

EXPERIMENT 8

Write a Python Program to Implement Depth-First Search (DFS) Algorithm

Aim

To develop a Python program to implement the **Depth-First Search (DFS)** algorithm for traversing a graph by exploring each branch completely before backtracking.

Objective

- To understand the concept of **Depth-First Search (DFS)**.
- To implement DFS using recursion.
- To traverse all nodes of a graph without revisiting any node.
- To study graph traversal techniques used in Artificial Intelligence.

Theory

Depth-First Search (DFS) is an **uninformed search algorithm** used to traverse or search graphs and trees. Unlike Breadth-First Search, DFS explores one branch completely before moving to another branch. It uses the **Stack (LIFO - Last In First Out)** principle, either explicitly using a stack or implicitly through recursion.

The basic working of DFS is:

- Start from the source node.
- Mark the source node as visited.
- Visit one unvisited adjacent node.
- Continue moving deeper until no unvisited nodes remain.
- Backtrack to the previous node.
- Repeat the process until all nodes have been visited.

DFS is widely used in Artificial Intelligence, pathfinding, puzzle solving, and graph-based applications.

Algorithm

Step 1: Create a graph using an adjacency list.

Step 2: Select the starting node.

Step 3: Mark the starting node as visited.

Step 4: Display the current node.

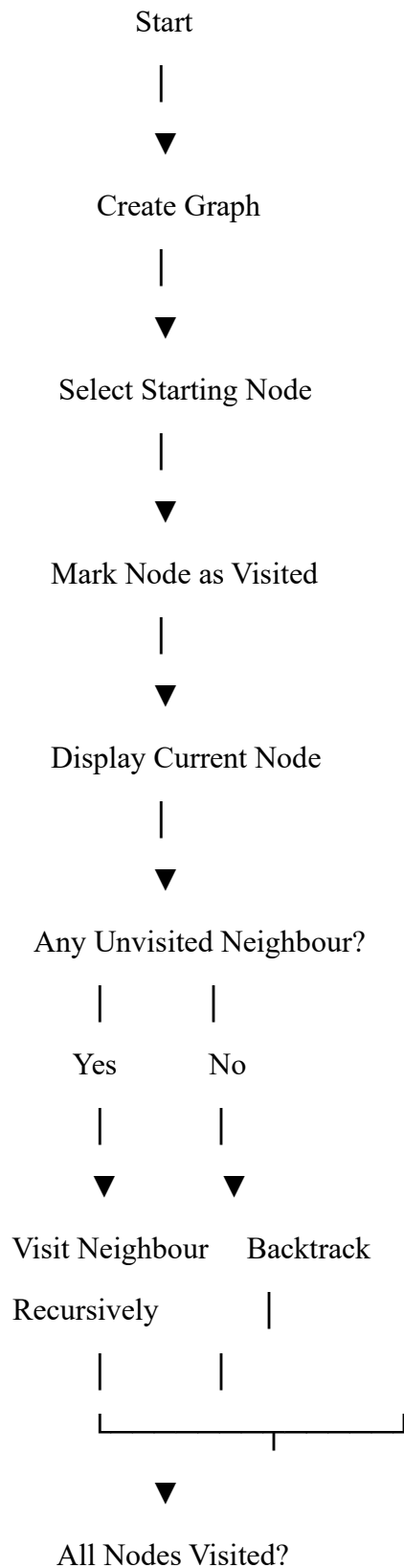
Step 5: Visit one unvisited adjacent node recursively.

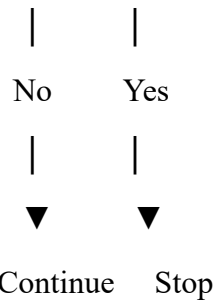
Step 6: Continue exploring deeper until no unvisited neighbors remain.

Step 7: Backtrack to the previous node.

Step 8: Repeat until all nodes have been visited.

Flowchart





Python Program

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': ['F'],  
    'F': []  
}  
  
visited = set()  
  
def dfs(node):  
  
    if node not in visited:  
  
        print(node, end=" ")  
  
        visited.add(node)  
  
        for neighbour in graph[node]:  
            dfs(neighbour)  
  
print("Depth First Traversal:")
```

dfs('A')

Sample Output

Depth First Traversal:

A B D E F C

Explanation

Consider the graph:

- A is connected to B and C.
- B is connected to D and E.
- C is connected to F.
- E is connected to F.

The DFS traversal starts from node A.

Traversal order:

A → B → D → E → F → C

The algorithm first explores the leftmost branch completely before returning (backtracking) to explore the remaining branches.

Advantages

- Requires less memory than Breadth-First Search.
- Simple and easy to implement.
- Suitable for deep search problems.
- Can be implemented using recursion or a stack.
- Efficient for exploring large graphs.

Disadvantages

- Does not always find the shortest path.
- Can get trapped in deep branches if cycles are not handled.
- Requires maintaining a visited list to avoid infinite loops.

Applications

- Artificial Intelligence search problems.

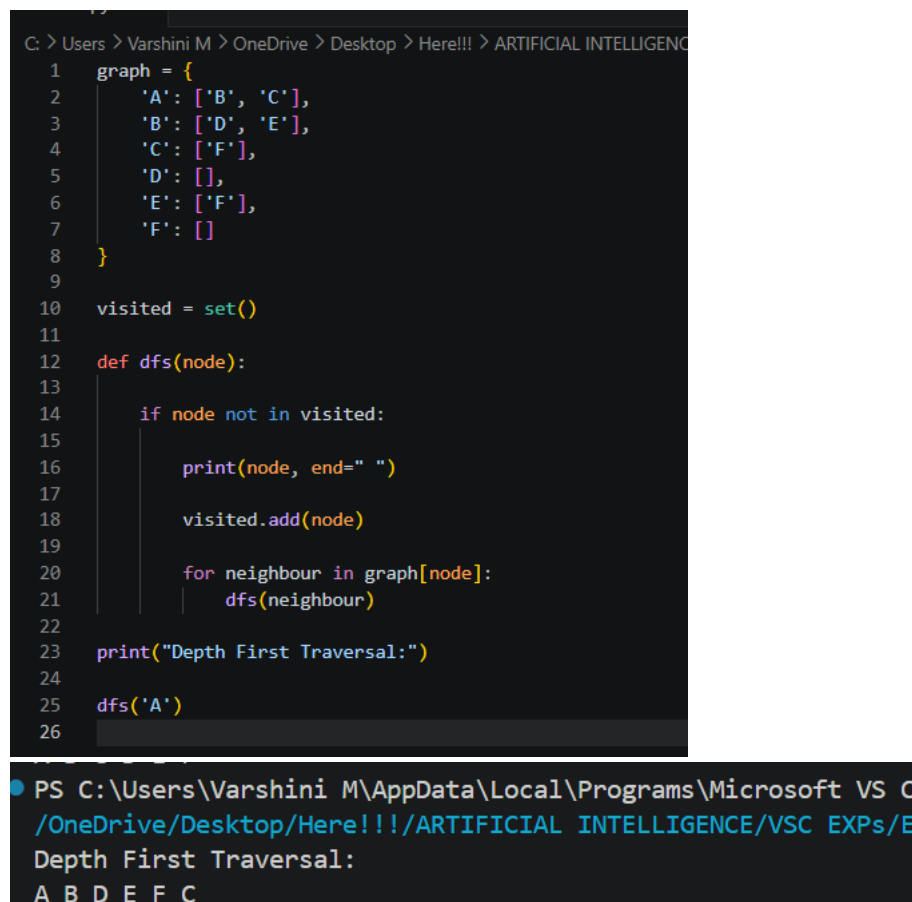
- Maze solving.
- Puzzle solving.
- Pathfinding.
- Topological sorting.
- Cycle detection in graphs.
- Network analysis.

Result

The Python program successfully implemented the **Depth-First Search (DFS)** algorithm and traversed all nodes of the graph by exploring one branch completely before backtracking.

Conclusion

The **Depth-First Search (DFS)** algorithm is a fundamental graph traversal technique used in Artificial Intelligence. It explores nodes deeply before backtracking, making it suitable for applications such as maze solving, puzzle solving, and graph analysis. The use of recursion or a stack enables efficient traversal while ensuring every reachable node is visited exactly once. This experiment demonstrates the practical implementation and importance of DFS in solving search and traversal problems.



```

C: > Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE
1  graph = {
2      'A': ['B', 'C'],
3      'B': ['D', 'E'],
4      'C': ['F'],
5      'D': [],
6      'E': ['F'],
7      'F': []
8  }
9
10 visited = set()
11
12 def dfs(node):
13
14     if node not in visited:
15
16         print(node, end=" ")
17
18         visited.add(node)
19
20         for neighbour in graph[node]:
21             dfs(neighbour)
22
23 print("Depth First Traversal:")
24
25 dfs('A')
26
PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS C
/OneDrive/Desktop/Here!!!/ARTIFICIAL INTELLIGENCE/VSC EXPs/E
Depth First Traversal:
A B D E F C

```